

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

CASE STUDY

Code of Conduct:

I _____ CH.Bhavya sri/192411249 _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Case Study

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-	Organized	Limited	Poor clarity;

		structured, and easy to understand	and understandab le presentation	clarity and structure	difficult to understand
--	--	--	---	--------------------------	----------------------------

1: MD5 Password Storage Weaknesses and Secure SHA-Based Solution

Introduction

A software company stores user passwords using the MD5 hashing algorithm. After a database breach, attackers crack many passwords because MD5 is an outdated and insecure hashing algorithm. Modern password security requires stronger hashing methods with salting and multiple iterations.

Weaknesses of MD5

MD5 generates a 128-bit hash, but it has several security weaknesses.

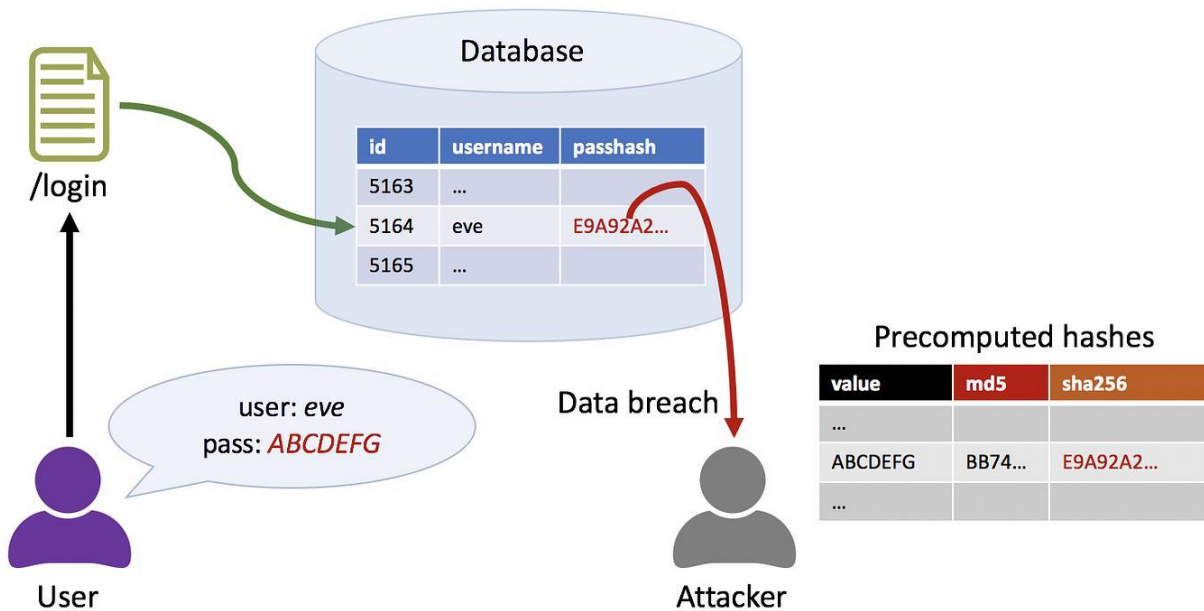
First, MD5 is vulnerable to collision attacks, where two different inputs can produce the same hash value. Although password cracking usually relies more on brute force and rainbow tables than collisions, this weakness shows that MD5 is no longer cryptographically secure.

Second, MD5 is extremely fast. Modern GPUs can compute billions of MD5 hashes every second, allowing attackers to quickly test millions of password guesses.

Third, if passwords are stored without unique salts, identical passwords produce identical hashes. This allows attackers to identify users with the same password and use precomputed rainbow tables to recover passwords rapidly.

Fourth, MD5 has been considered broken for cryptographic security for many years and is no longer recommended for password storage.

How Attackers Crack MD5 Passwords



After obtaining the password database, attackers compare stored MD5 hashes with precomputed rainbow tables. They also perform brute-force and dictionary attacks using high-speed GPUs. Since MD5 is very fast, weak passwords are recovered within seconds or minutes.

More Secure SHA-Based Approach

A better solution is to use SHA-256 together with salting and multiple hashing iterations. Each user's password should receive a unique random salt before hashing. The salt is stored with the hash.

For example:

- Password: Password123
- Salt: X7mP9Q
- Stored Value: SHA-256(Salt + Password)

Using thousands or hundreds of thousands of hashing iterations makes password cracking significantly slower.

In real-world applications, algorithms such as **PBKDF2-HMAC-SHA256** are recommended because they repeatedly apply SHA-256 to increase computational cost.

Comparison

Feature	MD5	SHA-256 with Salt
Hash Size	128-bit	256-bit
Collision Resistance	Weak	Strong
Rainbow Table Protection	No	Yes
Password Cracking Speed	Very Fast	Much Slower
Modern Security	Not Recommended	Recommended

Additional Security Improvements

Strong password policies should require long and complex passwords. Multi-factor authentication should be enabled to reduce the impact of password theft. Login attempts should be rate-limited, and password databases should be encrypted with proper access controls.

Conclusion

MD5 is unsuitable for password storage because it is fast and vulnerable to modern attacks. SHA-256 combined with unique salts and repeated hashing provides significantly stronger protection.

Digital Signature Flaws in Banking Transactions

Introduction

A banking application uses digital signatures to verify transactions. However, unauthorized transactions are being approved, indicating weaknesses in the implementation rather than in the digital signature concept itself.

Possible Flaws

One possible problem is improper signature verification. If the server does not verify every transaction before processing it, forged transactions may be accepted.

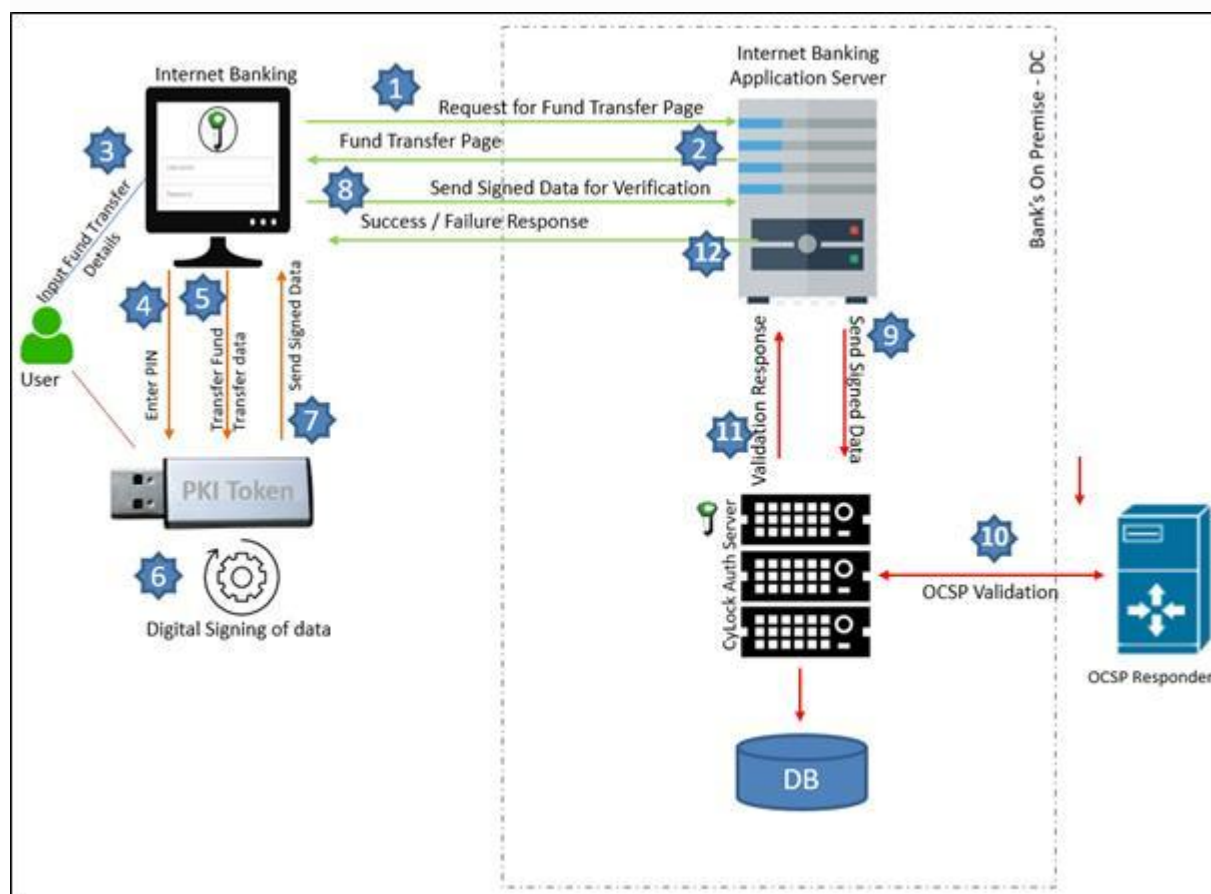
Another problem is weak private key protection. If attackers steal users' private keys from infected devices, they can create valid signatures.

Some implementations sign only part of the transaction instead of all transaction details. For example, signing only the amount while leaving the destination account unsigned allows attackers to modify important information.

Weak or outdated algorithms such as MD5-based signatures also reduce security.

Replay attacks are another possibility. If previously signed transactions can be submitted again, duplicate unauthorized payments may occur.

How Secure Digital Signatures Should Work



The customer creates a transaction, hashes the complete transaction details, signs the hash with a private key, and sends both the transaction and signature. The bank verifies the signature using the customer's public key before processing the payment.

Recommended Improvements

The system should use modern algorithms such as RSA-3072, ECDSA, or Ed25519.

Every transaction should include all critical information in the signed data, including

- Amount
- Receiver account
- Timestamp
- Transaction ID

Private keys should be stored inside secure hardware such as TPMs or Hardware Security Modules.

Each transaction should contain a unique nonce to prevent replay attacks.

Multi-factor authentication should protect high-value transactions.

Benefits

Security Measure	Benefit
RSA/ECDSA	Strong signatures
Secure Key Storage	Prevents key theft
Timestamp	Prevents replay
Transaction ID	Prevents duplication
MFA	Additional protection

Conclusion

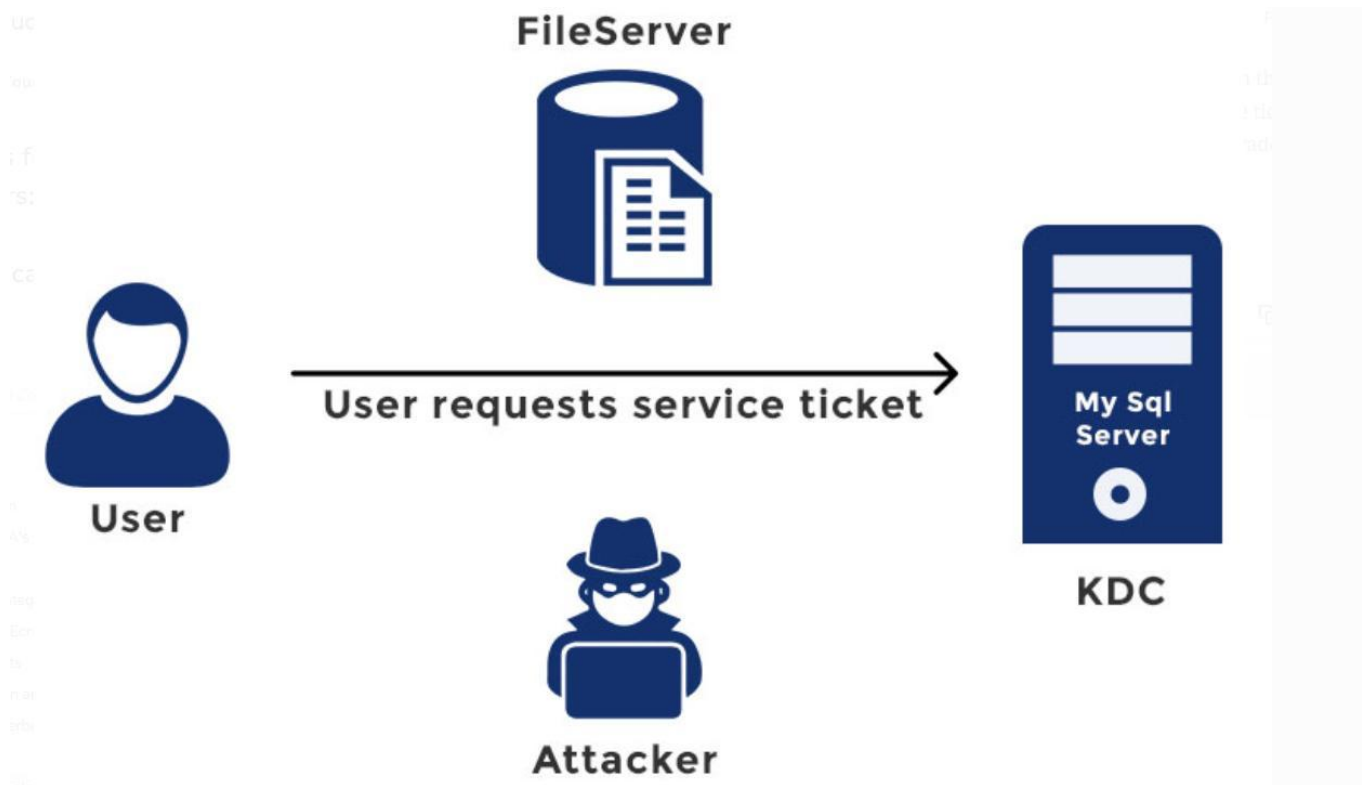
Proper verification, secure key management, and modern signature standards significantly reduce unauthorized banking transactions.

3: Kerberos Replay Attack

Introduction

Kerberos authenticates users through tickets issued by a trusted Key Distribution Center. Despite this protection, replay attacks may succeed if tickets are reused improperly.

How Replay Attack Happens



An attacker captures a valid authentication ticket from the network. If the ticket remains valid and the server does not properly verify freshness, the attacker resends the same ticket and gains unauthorized access.

Reasons Replay Attacks Succeed

Long ticket lifetimes increase the attack window.

Servers that fail to verify timestamps allow old tickets to be reused.

Unsynchronized system clocks may weaken timestamp validation.

Lack of authenticators or session validation also increases risk.

Enhancements to Prevent Replay Attacks

Ticket lifetimes should be kept short.

Every authentication request should include a fresh authenticator containing a timestamp.

Servers should reject duplicate authenticators.

Clocks across the network should remain synchronized using secure NTP.

Mutual authentication should verify both client and server identities.

Session keys should be regenerated whenever necessary.

Improved Authentication Process

- Client requests a Ticket Granting Ticket.
- Authentication Server verifies credentials.
- Client receives a time-limited ticket.
- Client sends the ticket and a fresh authenticator.
- Server validates the timestamp and session key.
- Duplicate requests are rejected.

Comparison

Feature	Weak Kerberos	Improved Kerberos
Ticket Lifetime	Long	Short
Timestamp Check	Weak	Strong
Duplicate Detection	No	Yes
Mutual Authentication	Optional	Enabled

Conclusion

Replay attacks become much more difficult when tickets expire quickly, authenticators are verified, and timestamps remain synchronized.

4: Simple Hash vs HMAC

Introduction

A secure communication system uses only a simple hash to verify message integrity. Attackers modify messages without detection because the system lacks authentication.

Why Simple Hash Fails

A hash function only detects accidental changes.

Since no secret key is involved, anyone can modify both the message and its hash.

For example:

Original:

Transfer ₹500

Hash:

ABC123

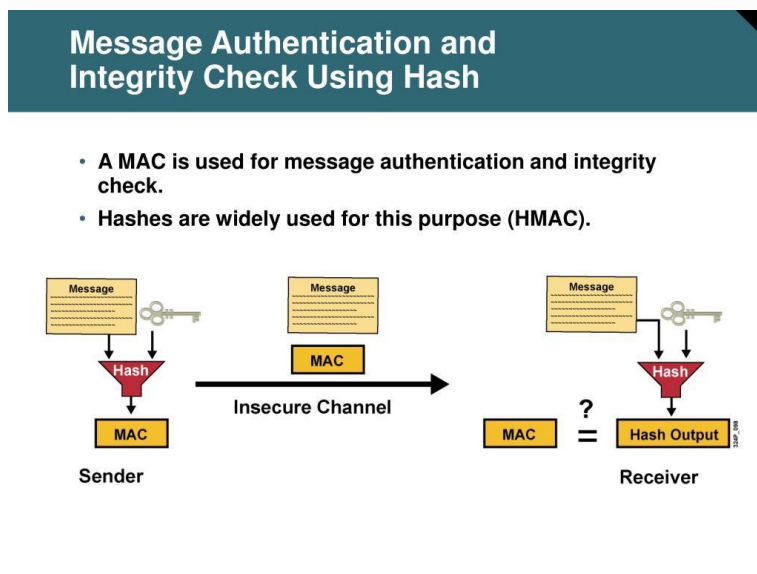
Attacker changes the message to:

Transfer ₹5000

The attacker simply computes a new hash and sends both together.

The receiver cannot determine whether the sender or attacker created the new hash.

Attack Illustration



A simple hash allows anyone to recompute the hash after modifying a message, while HMAC combines a secret key with the hash so only authorized parties can generate a valid authentication code.

Why HMAC Solves the Problem

HMAC combines:

Secret Key

Message

Secure Hash Function (SHA-256)

The sender computes:

HMAC = HMAC(Key, Message)

The receiver uses the same secret key to verify it.

An attacker without the key cannot generate a valid HMAC.

Security Comparison

Feature	Simple Hash	HMAC
Secret Key	No	Yes
Integrity	Yes	Yes
Authentication	No	Yes
Tamper Protection	Weak	Strong

- Applications of HMAC**
- HMAC is widely used in:
 - Banking APIs
 - HTTPS
 - Cloud services
 - JWT authentication

Additional Improvements

The system should use SHA-256 or SHA-3 inside HMAC, rotate secret keys periodically, and establish secure key distribution mechanisms.

Conclusion

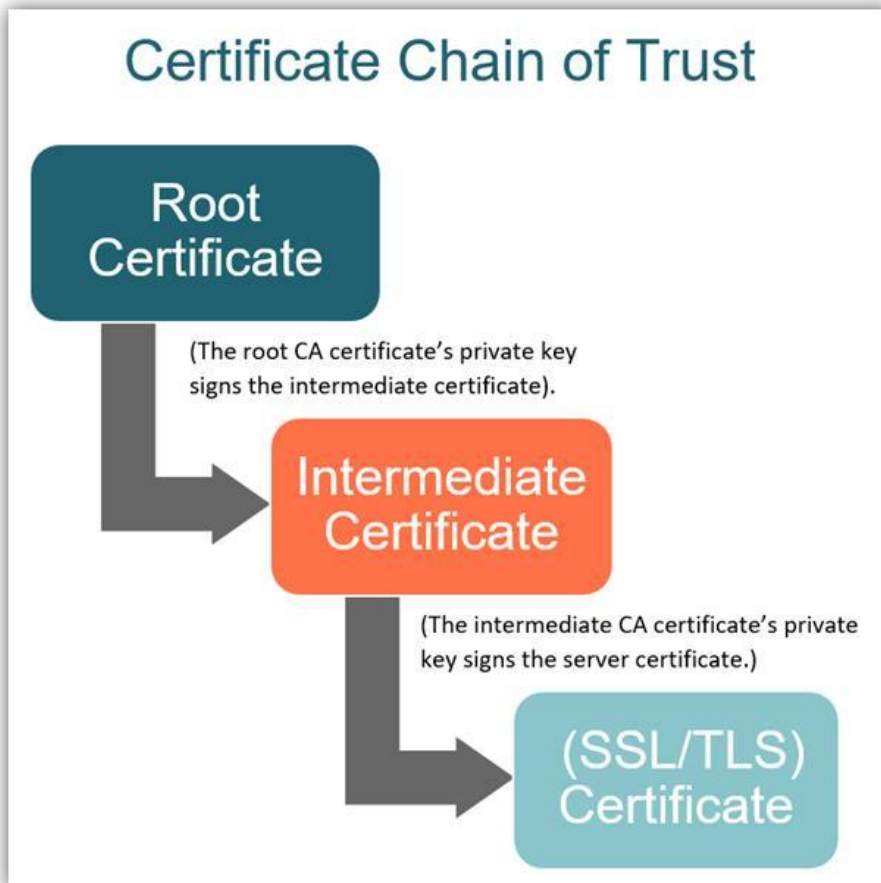
A simple hash cannot authenticate message origin, while HMAC provides both integrity and authentication, making message tampering much more difficult.

5. X.509 Certificate Trust Validation Errors

Introduction

An organization uses X.509 certificates for secure communication, but users frequently receive trust validation errors. These problems usually result from certificate management issues rather than encryption failures.

How X.509 Trust Chain Works



A trusted Root Certificate Authority signs an Intermediate CA certificate, which signs the Server Certificate. Clients verify this chain before trusting the server.

Possible Causes

One common cause is an expired certificate. Browsers reject certificates that have passed their expiration date.

Missing intermediate certificates also break the trust chain.

Self-signed certificates are trusted only when manually installed.

Incorrect system time can make valid certificates appear expired.

Hostname mismatches occur when the certificate's domain does not match the website being accessed.

Certificates may also be revoked after compromise.

Solutions

The organization should renew certificates before expiration.

Complete certificate chains must be installed correctly.

Certificates should be obtained from trusted Certificate Authorities.

Automatic certificate management tools can simplify renewal.

Online Certificate Status Protocol (OCSP) and Certificate Revocation Lists (CRLs) should be checked regularly.

Servers should be configured correctly for HTTPS.

Certificate Management Best Practices

Best Practice	Benefit
Automatic Renewal	Prevents expiration
Trusted CA	Ensures browser trust
Full Chain Installation	Prevents validation errors
OCSP/CRL Checking	Detects revoked certificates

Benefits of Proper Certificate Management

Proper certificate management provides strong authentication, encrypted communication, trusted connections, and reduced security warnings.

Conclusion

Most X.509 trust errors result from poor certificate management. Maintaining valid certificates, complete trust chains, accurate system time, and proper revocation checking ensures secure authentication and reliable encrypted communication.